



UNIVERSITÉ  
LAVAL

---

## Laboratoire 2

---

Réalisé par  
Corinne Dumais (111 263 065)

Travail présenté à  
M. Mir Abolfazl Mostafavi

Dans le cadre du cours  
GMT-7035 Structures de données géométriques et algorithmiques en SIG

Département des Sciences géomatiques  
Faculté de foresterie, de géographie et de géomatique  
Université Laval  
20 mars 2023

# Table des matières

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                           | <b>2</b>  |
| <b>2</b> | <b>Description des algorithmes</b>                                            | <b>2</b>  |
| 2.1      | Initialisation du triangle universel ( <i>frame</i> ) . . . . .               | 2         |
| 2.2      | Position d'un point par rapport à un segment ( <i>determinant</i> ) . . . . . | 2         |
| 2.3      | Recherche du triangle contenant un point ( <i>walk</i> ) . . . . .            | 3         |
| 2.4      | Insertion d'un point de le maillage ( <i>insert</i> ) . . . . .               | 3         |
| 2.5      | Optimisation des triangles ( <i>optim</i> ) . . . . .                         | 4         |
| 2.6      | Échange des diagonales ( <i>swap</i> ) . . . . .                              | 4         |
| 2.7      | Traçage de la triangulation Delaunay ( <i>tracage_delaunay</i> ) . . . . .    | 5         |
| 2.8      | Traçage du diagramme de Voronoi ( <i>tracage_voronoi</i> ) . . . . .          | 5         |
| 2.9      | Voisin Plus Proche ( <i>voisin_plus_proche</i> ) . . . . .                    | 5         |
| 2.10     | Voisins immédiats ( <i>voisins_immediats</i> ) . . . . .                      | 5         |
| <b>3</b> | <b>Guide utilisateur</b>                                                      | <b>6</b>  |
| 3.1      | Interface générale . . . . .                                                  | 6         |
| 3.2      | Description des onglets et options . . . . .                                  | 6         |
| 3.3      | Étapes d'utilisation . . . . .                                                | 8         |
| <b>4</b> | <b>Résultats</b>                                                              | <b>9</b>  |
| <b>5</b> | <b>Conclusion</b>                                                             | <b>12</b> |

# 1 Introduction

La triangulation Delaunay est une méthode qui permet d'obtenir des triangles optimisés afin de former un maillage triangulaire. Ce type de structure peut par exemple servir à représenter le relief d'un terrain. En utilisant des structures de données computationnellement efficace, il est également possible de facilement faire des opérations d'analyse spatiale dans un tel maillage.

L'objectif de ce laboratoire était donc de produire une application permettant à un utilisateur de former une triangulation Delaunay et insérant lui-même des points, puis faire afficher le diagramme de Voronoi et effectuer d'autres traitements comme la recherche de voisin le plus proche ou des voisins immédiats.

## 2 Description des algorithmes

Note : En ce qui a trait aux noms donnés aux fonctions, je suis restée le plus près possible des noms donnés dans l'énoncé, mais je les ai adapté pour respecter les bonnes pratiques de nommage de variable en Python (format *nom\_de\_variable*).

### 2.1 Initialisation du triangle universel (*frame*)

Les coordonnées des trois sommets du triangle universel sont déterminées aléatoirement, mais à l'intérieur d'intervalles donnés afin de garantir que le triangle sera suffisamment gros dans l'interface pour contenir une bonne quantité de points. Les coordonnées sont ensuite stockées et le polygone est dessiné.

### 2.2 Position d'un point par rapport à un segment (*determinant*)

Cette fonction est utilisée pour déterminer si un point  $p$  est à gauche d'un segment dont les extrémités sont  $p_1$  et  $p_2$ . On calcule le déterminant suivant :

$$det = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_p & y_p & 1 \end{vmatrix}$$

Si le déterminant est non-négatif, la fonction retourne vrai car le point est du bon côté du segment. Sinon elle retourne faux.

### 2.3 Recherche du triangle contenant un point (*walk*)

Cette fonction prend un point  $p$  et retourne l'ID du triangle dans lequel il se trouve. La fonction commence au triangle 1, puis utilise itérativement la fonction *determinant* sur chacun des côtés du triangles pour voir si le point est du bon côté. Si déterminant retourne faux, alors la fonction reprend les calculs de déterminants sur le triangle adjacent dont le côté en commun est celui pour lequel le test à échoué. S'il n'y a pas de triangle ou si on est arrivé dans ce triangle par ce côté, on passe au prochain segment. Le triangle dans lequel se trouve le point est trouvé lorsque le test du déterminant réussit sur les trois côtés.

### 2.4 Insertion d'un point de le maillage (*insert*)

Cette fonction est utilisée une fois qu'on connaît le triangle dans lequel se trouve le point. On crée donc trois nouveaux triangles en reliant le nouveau point aux sommets du triangle contenant le point. La nouvelle configuration de ces triangles est stockée dans la structure TIN. L'un des nouveaux triangles va écraser la définition du triangle maintenant divisé, car il n'existe plus et les deux autres triangles vont se voir attribuer un numéro  $N+1$  et  $N+2$ ,  $N$  étant le nombre de triangle avant la nouvelle insertion. Finalement, les triangles adjacents à l'ancien triangle sont modifiés lorsque nécessaire, c'est-à-dire que certaines références à l'ancien triangle sont écrasées par les références aux nouveaux triangles. Les nouveaux triangles sont également ajoutés sur la pile pour ensuite faire l'optimisation.

## 2.5 Optimisation des triangles (*optim*)

La fonction d'optimisation fonctionne tant qu'il y a des éléments sur la pile. Elle en dépile un et vérifie s'il est un triangle valide Delaunay. S'il ne l'est pas, on inverse les diagonales. Ci-dessous est présenté un pseudo-code.

```
FONCTION optim
  TANT QUE pile pas vide
    t1 = dépile un triangle
    t2 = trouve le triangle adjacent au côté opposé au nouveau point
    SI pnc est dans le cercle englobant t1:
      échange les diagonales de t1 et t2
      efface les anciennes diagonales
      dessine les nouvelles diagonales
  FIN TANT QUE
```

## 2.6 Échange des diagonales (*swap*)

Cette fonction identifie tout d'abord dans la configuration problématique le nouveau point (np), les deux points communs (pc1 et pc2) et le point non-commun (pnc). Les côtés qui ne sont pas le côté commun aux deux triangles T1 et T2 sont identifiés T3, T4, T5, T6 et on trouve les identificateurs de ces triangles. On change ensuite la diagonale entre les deux points communs pour une diagonale entre le nouveau point et le point non-commun. On met ensuite à jour les définitions de tous les triangles affectés par ce changement.

```
FONCTION swap (T1, T2)
  trouve np, pc1, pc2, pnc
  trouve T3, T4, T5, T6
  Écrase les définitions de T1 et T2 avec leur nouvelle configuration
  Met à jour les triangles adjacents qui sont affectés (T4 et T6)
  Met T1 et T2 sur la pile
  retourne np, pnc, pc1, pc2
```

## 2.7 Traçage de la triangulation Delaunay (*tracage\_delaunay*)

L'algorithme global de triangulation Delaunay consiste à appeler séquentiellement les fonctions ci-haut tant que l'utilisateur continue d'insérer des points. Ci-dessous est un pseudo-code qui décrit ce processus.

```
FONCTION tracage_delaunay
    TANT QUE utilisateur insère des points
        Stocker le nouveau point p et le dessiner avec dessiner_nouveau_point()
        T = Trouver le triangle dans lequel se trouve p avec walk()
        Séparer T en 3 nouveaux triangles et les mettre sur la pile avec insert()
        Optimiser les triangles sur la pile avec optim()
    FIN TANT QUE
```

## 2.8 Traçage du diagramme de Voronoi (*tracage\_voronoi*)

Pour cette fonction, on calcule tout d'abord le centre circonscrit de chacun des triangles du maillage. Puis, pour chaque triangle, on relie son centre circonscrit à celui de ses voisins et on trace ces segments.

## 2.9 Voisin Plus Proche (*voisin\_plus\_proche*)

Tout d'abord, le triangle dans lequel se trouve le point inséré est identifié avec la fonction *walk()*. Puis, la distance euclidienne entre le point et chacun des trois sommets du triangle est calculé. Le sommet ayant la plus petite distance euclidienne avec le point est retourné comme étant le voisin le plus proche.

## 2.10 Voisins immédiats (*voisins\_immediats*)

Cette fonction prend le numéro  $n$  d'un sommet pour lequel on veut tout le voisinage immédiat. La fonction itère sur tous les triangles : pour chaque triangle, si le sommet  $n$  est parmi les sommets du triangle, alors les deux autres sommets du triangle sont ajoutés au voisinage. On encercle les sommets trouvés en bleu.

## 3 Guide utilisateur

### 3.1 Interface générale

La Figure 1 présente l'interface générale de l'application lors de son démarrage.

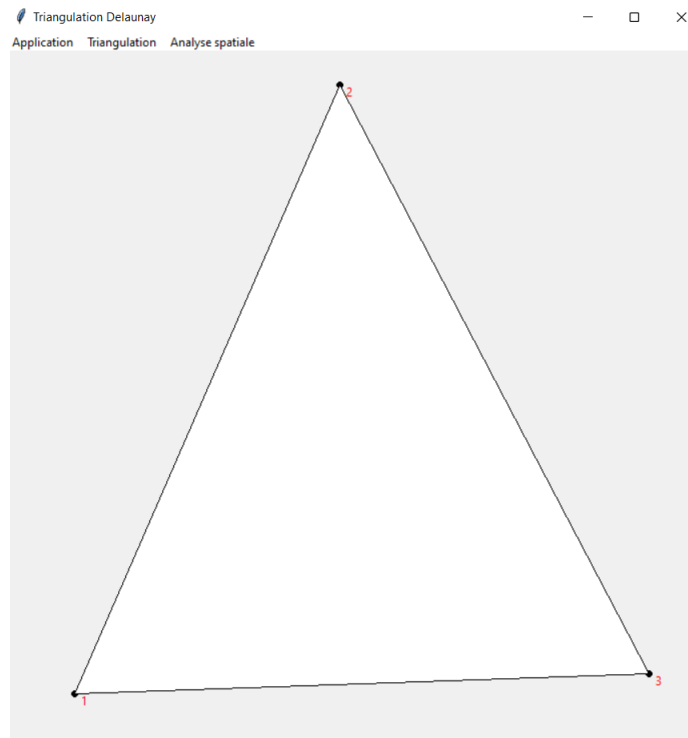


FIGURE 1 – Interface générale

### 3.2 Description des onglets et options

L'application a une interface simple avec trois onglets principaux dans la barre à options en haut de la fenêtre. Le premier onglet, illustré à la Figure 2, est « Application » et il comporte deux options : réinitialiser le programme avec « Réinitialiser » ou de quitter l'application avec « Exit ».

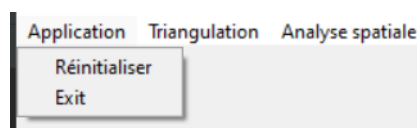


FIGURE 2 – Onglet Application

Le deuxième onglet, illustré à la Figure 3 est l'onglet « Triangulation ». Cet onglet permet de commencer à insérer des points pour la triangulation en cliquant sur « Triangulation Delaunay → Commencer ». L'option « Arrêter » est débloquée seulement lorsqu'une triangulation est en cours et, inversement, l'option de « Commencer » sera bloquée une fois que la triangulation est en cours. L'option pour tracer le diagramme Voronoi deviendra accessible une fois qu'une triangulation aura été complétée, c'est-à-dire qu'elle aura été commencée, puis arrêtée.

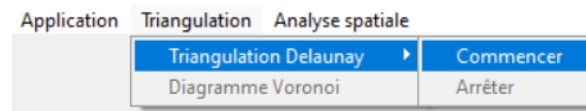


FIGURE 3 – Onglet Triangulation

Le troisième onglet, illustré à la Figure 4 est l'onglet « Analyse spatiale ». Similairement à plus haut, ces options sont bloquées puisqu'aucun point n'a été inséré. Ces options deviendront accessibles une fois que l'utilisateur a inséré des points et arrêté la triangulation.

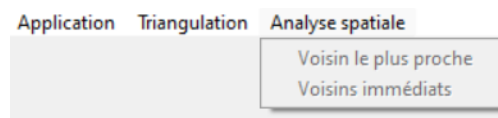


FIGURE 4 – Onglet Analyse spatiale

L'option « Voisin le plus proche » permet de cliquer dans le triangle universel et d'obtenir le sommet le plus proche. Le point est affiché comme un cercle vert plein et le sommet le plus proche est encerclé en vert (voir Figure 5).

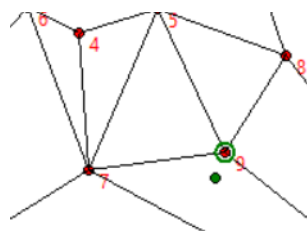


FIGURE 5 – Affichage du voisin le plus proche

L'option « Voisins immédiats » permet de sélectionner un sommet par son numéro et d'obtenir tous ses voisins immédiats. Les voisins sont encerclés en bleu (voir Figure 6).

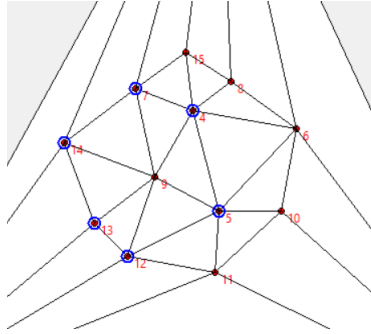


FIGURE 6 – Affichage des voisins immédiats.

### 3.3 Étapes d'utilisation

Les étapes d'utilisation à suivre sont les suivantes :

1. Lancer l'application.
2. Activer l'insertion de points en cliquant sur *Triangulation*  $\rightarrow$  *Triangulation Delaunay*  $\rightarrow$  *Commencer*.
3. Insérer le nombre de points désiré dans le triangle. Les clics à l'extérieur du triangle sont ignorés.
4. Désactiver l'insertion de points en cliquant sur *Triangulation*  $\rightarrow$  *Triangulation Delaunay*  $\rightarrow$  *Arrêter*.
5. Produire le diagramme de Voronoi en cliquant sur *Triangulation*  $\rightarrow$  *Diagramme Voronoi*.
6. Si désiré, effectuer des opérations d'analyse spatiale via les options de l'onglet *Analyse spatiale*.
7. À tout point, on peut réinitialiser la triangulation afin de revenir à l'étape 2.

## 4 Résultats

La Figure 7 présente un exemple de résultat produit avec l'insertion de 70 points.

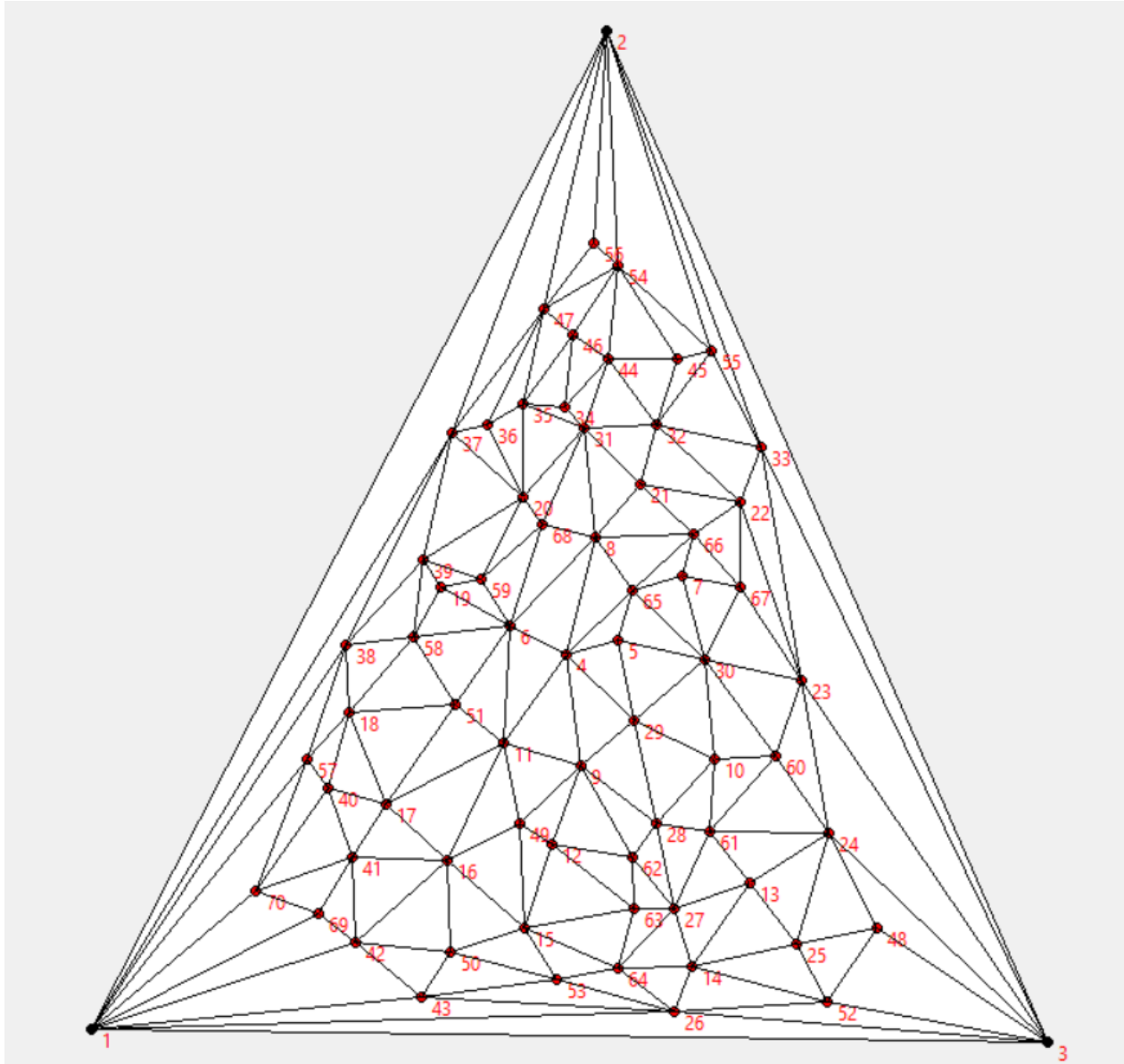


FIGURE 7 – Triangulation Delaunay avec 70 points

La Figure 8 présente la même triangulation, cette fois-ci avec 15 points et le diagramme de Voronoi affiché.

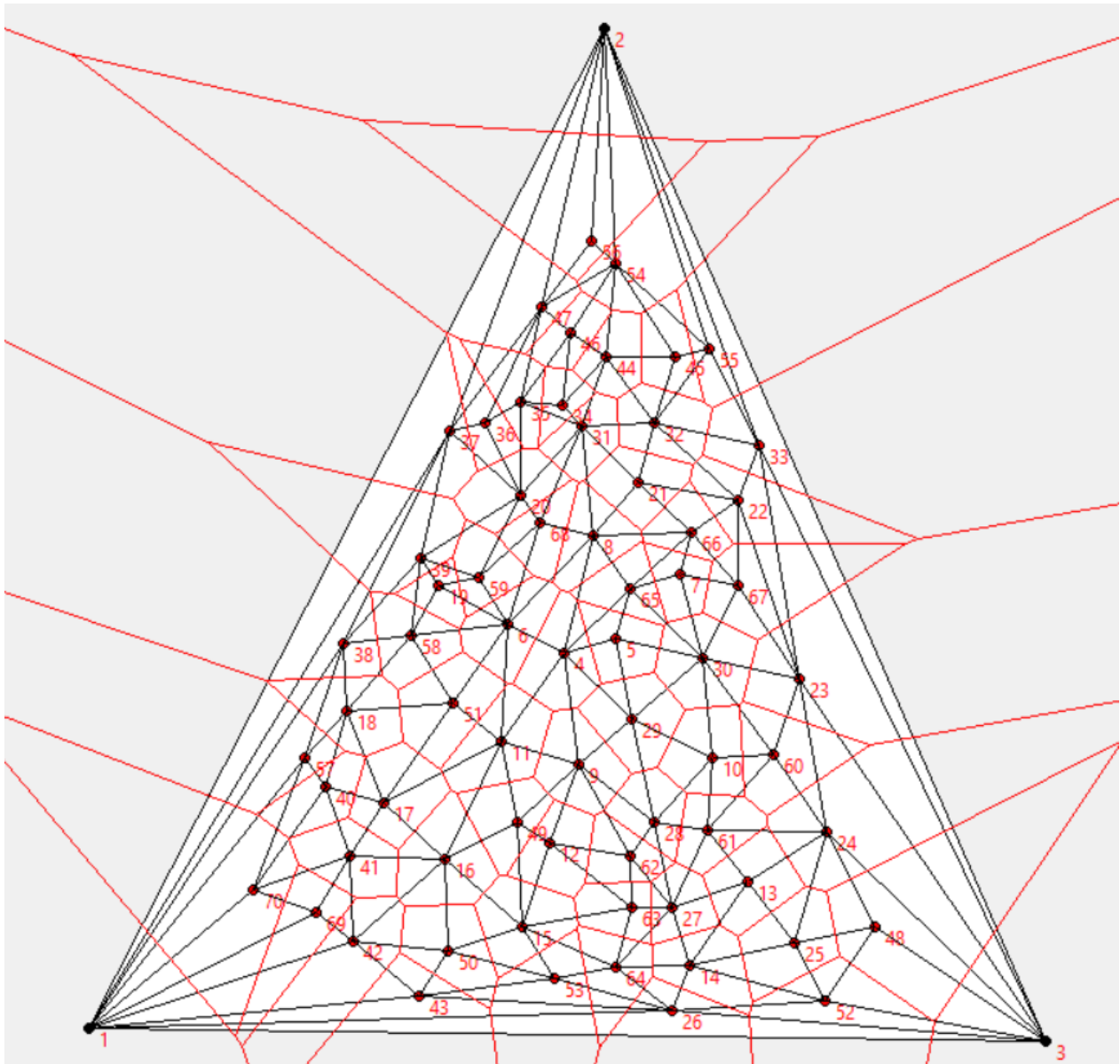


FIGURE 8 – Diagramme Voronoi de la triangulation 7

On voit que chacun des points est effectivement dans une cellule de Voronoi.

La Figure 9 présente une autre triangulation, cette fois-ci de 20 points, après laquelle on a voulu trouver le sommet le plus proche du cercle plein vert, puis les voisins immédiats du sommet 5. La diagramme de Voronoi n'a pas été affiché afin de faciliter la visualisation des fonctions d'analyse spatiale.

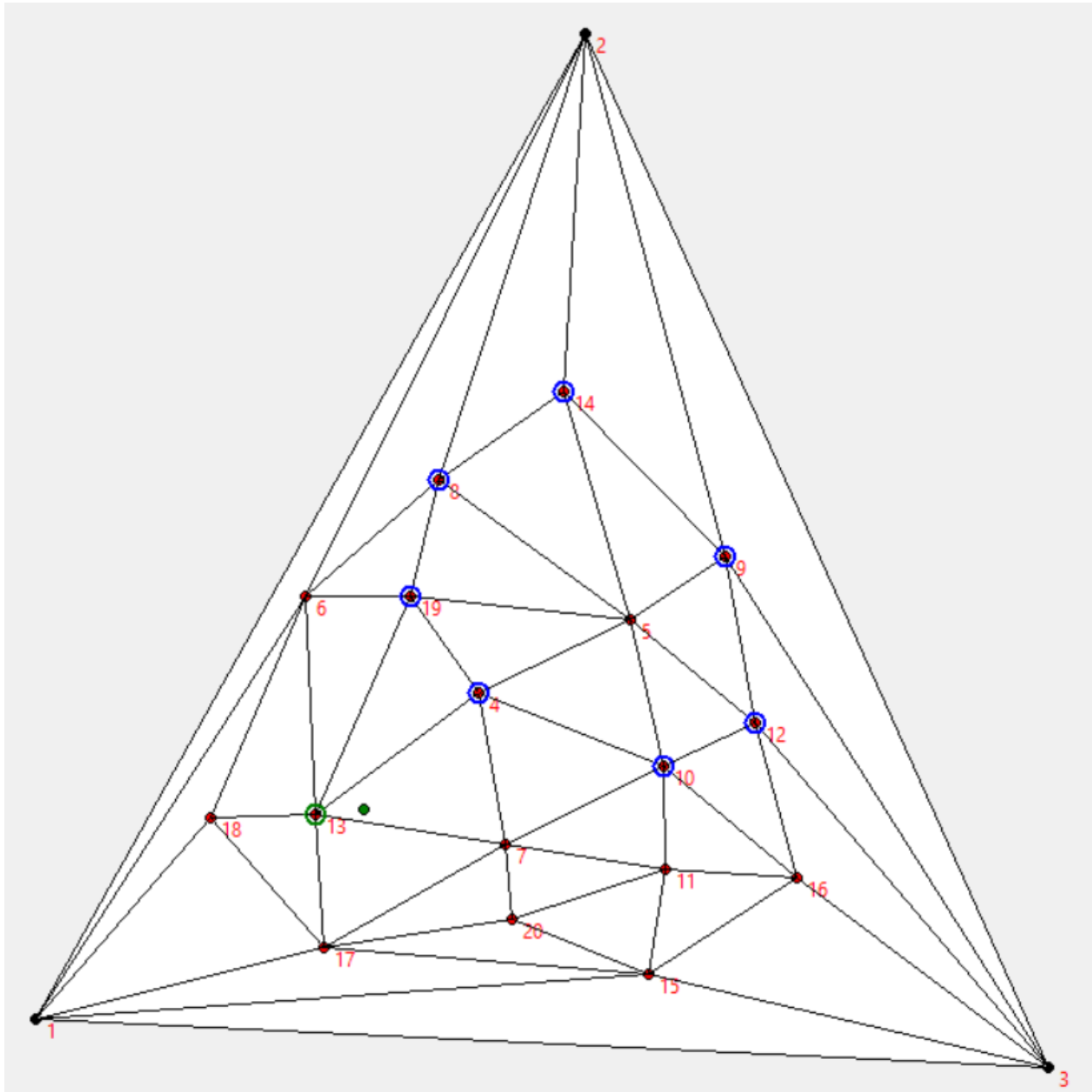


FIGURE 9 – Triangulation Delaunay avec 70 points

## 5 Conclusion

En conclusion, l'objectif de produire une application de base effectuant de la triangulation Delaunay a été atteint. L'application effectue toutes les fonctionnalités demandées et est simple à comprendre pour un utilisateur. Quelques améliorations pourraient être apportés dans un travail plus poussé. Par exemple, il serait intéressant d'importer des points à partir d'un fichier et de produire la triangulation Delaunay. Il serait également intéressant de pouvoir exporter/sauvegarder les résultats afin que l'utilisateur puisse les réutiliser.

## Annexe A : Code de delaunay.py

```

import tkinter as tk
from tkinter import *
from tkinter import simpledialog
from typing import List, Tuple
import numpy as np
from numpy.random import randint

class Vertex:
    """ -----
    Classe qui modélise un sommet.
    ----- """

    def __init__(self, x, y, number=None):
        self.number = number
        self.x = x
        self.y = y

class Triangle:
    """ -----
    Classe qui modélise un triangle.
    ----- """

    def __init__(self, ID: int, vertices: List[Vertex], adjacents: List[int]):
        self.ID = ID
        self.vertices = vertices
        self.adjacents = adjacents
        self.center = None

class DelaunayTriangulationApp:
    """ -----
    Classe qui implémenter l'application pour la triangulation Delaunay.
    ----- """

    def __init__(self, height: int, width: int, displayNumbers=False):
        self.height = height
        self.width = width
        self.displayNumbers = displayNumbers

        self.next_vertex_id = 1
        self.next_triangle_id = 1

        self.TIN = {}
        self.lines = {}
        self.text_ids = []
        self.stack = []
        self.new_point = None
        self.universal = None
        self.delaunay = False

        self.root = tk.Tk()
        self.canvas = tk.Canvas(self.root, width=self.width, height=self.height)
        self.canvas.pack()

        self.launch()

    def launch(self):
        """ -----
        Nom: launch
        Auteur: Corinne Dumais
        Description: Lance l'exécution de la fenêtre de l'application et formate le menu des options.
        Date: 2023-13-03
        """

```

```

Version: 1.00
----- """
# Paramètres de la fenêtre d'application
self.root.title("Triangulation Delaunay")
self.root.minsize(self.height, self.width)
self.root.maxsize(self.height, self.width)

# On crée le menu
self.menubar = Menu(self.root)
self.root.config(menu=self.menubar)

self.app_menu = Menu(self.menubar, tearoff=False)
self.triangulation_menu = Menu(self.menubar, tearoff=False)
self.analysis_menu = Menu(self.menubar, tearoff=False)

self.sub_draw = Menu(self.triangulation_menu, tearoff=False)
self.sub_draw.add_command(label='Commencer', command=self.tracage_delaunay)
self.sub_draw.add_command(label='Arrêter', command=self.stop_delaunay)
self.sub_draw.entryconfigure(1, state='disabled')

self.app_menu.add_command(label='Réinitialiser', command=self.clear)
self.app_menu.add_command(label='Exit', command=self.root.destroy)

self.triangulation_menu.add_cascade(label='Triangulation Delaunay', menu=self.sub_draw)
self.triangulation_menu.add_command(label='Diagramme Voronoi', command=self.tracage_voronoi)
self.triangulation_menu.entryconfigure(1, state='disabled')

self.analysis_menu.add_command(label='Voisin le plus proche', command=self.voisin_plus_proche)
self.analysis_menu.add_command(label='Voisins immédiats', command=self.voisins_immediats)
self.analysis_menu.entryconfigure(0, state='disabled')
self.analysis_menu.entryconfigure(1, state='disabled')

self.menubar.add_cascade(label="Application", menu=self.app_menu)
self.menubar.add_cascade(label="Triangulation", menu=self.triangulation_menu)
self.menubar.add_cascade(label="Analyse spatiale", menu=self.analysis_menu)

self.frame()
self.root.mainloop()

def clear(self):
    """ -----
    Nom: clear
    Auteur: Corinne Dumais
    Description: Réinitialise le triangle universel et l'exécution de l'application.
    Date: 2023-13-03
    Version: 1.00
    ----- """
    # Réinitialisation des variables de triangulation
    self.next_vertex_id = 1
    self.next_triangle_id = 1
    self.TIN = {}
    self.lines = {}
    self.text_ids = []
    self.stack = []
    self.new_point = None
    self.delaunay = False
    self.universal = None

    # Réinitialisation du canvas
    self.canvas.destroy()
    self.canvas = tk.Canvas(self.root, width=self.width, height=self.height)
    self.canvas.pack()

    # Réinitialisation du menu des options
    self.analysis_menu.entryconfigure(0, state='disabled')
    self.analysis_menu.entryconfigure(1, state='disabled')
    self.triangulation_menu.entryconfigure(1, state='disabled')
    self.sub_draw.entryconfigure(0, state='normal')

```

```

self.sub_draw.entryconfigure(1, state='disabled')

self.launch()

def frame(self):
    """ -----
    Nom: frame
    Auteur: Corinne Dumais
    Description: Formation du triangle universel.
    Date: 2023-13-03
    Version: 1.00
    ----- """
    vertices = []

    # Affichage des sommets du triangle universel
    for x, y in [(randint(20, 80), randint(620, 680)), (randint(320, 380), randint(20, 80)),
                 (randint(620, 680), randint(620, 680))]:
        self.canvas.create_oval(x - 3, y - 3, x + 3, y + 3, fill='black')
        self.canvas.create_text(x + 7, y + 7, text=str(self.next_vertex_id), anchor="w", fill='red')
        vertices.append(Vertex(x, y, self.next_vertex_id))
        self.next_vertex_id += 1

    # On stocke le triangle universel
    self.TIN[self.next_triangle_id] = Triangle(self.next_triangle_id, vertices, [0, 0, 0])
    self.next_triangle_id += 1

    # On dessine le triangle universel
    vertex1, vertex2, vertex3 = vertices
    self.universal = self.canvas.create_polygon(vertex1.x, vertex1.y, vertex2.x, vertex2.y, vertex3.x, vertex3.y,
                                                outline='black', fill='white')

    @staticmethod
    def determinant(p: Vertex, p1: Vertex, p2: Vertex) -> bool:
        """ -----
        Nom: determinant
        Auteur: Corinne Dumais
        Description: Calcule le déterminant pour déterminer si un point est à gauche d'un segment. Retourne faux si le
        déterminant est négatif, vrai sinon.
        Date: 2023-13-03
        Version: 1.00
        ----- """
        determinant = np.linalg.det([[p1.x, p1.y, 1], [p2.x, p2.y, 1], [p.x, p.y, 1]])
        return determinant >= 0

    def walk(self, p: Vertex) -> int:
        """ -----
        Nom: walk
        Auteur: Corinne Dumais
        Description: Trouve le triangle qui contient le nouveau point. Retourne l'ID du triangle.
        Date: 2023-13-03
        Version: 1.00
        ----- """
        # Commencer avec le triangle en position 1
        old_id = None
        next_id = 1
        found = False

        while not found:
            # Itérer sur chaque segment
            for i in range(3):
                success = self.determinant(p, self.TIN[next_id].vertices[i], self.TIN[next_id].vertices[(i + 1) % 3])
                if not success and self.TIN[next_id].adjacents[(i + 2) % 3] != 0 and self.TIN[next_id].adjacents[(i + 2) % 3] != 0:
                    old_id = next_id
                    next_id = self.TIN[old_id].adjacents[(i + 2) % 3]
                    break
            if i == 2:
                found = True

```

```

    return next_id

def dessiner_nouveau_point(self):
    """ -----
    Nom: draw_new_point
    Auteur: Corinne Dumais
    Description: Attend le clic de l'utilisateur et dessine le nouveau point.
    Date: 2023-13-03
    Version: 1.00
    ----- """

    def handle_click(event):
        if self.delaunay:
            if self.point_inside_universal(event.x, event.y, self.canvas.coords(self.universal)):
                x, y = event.x, event.y
                self.canvas.create_oval(x - 3, y - 3, x + 3, y + 3, fill='red')
                self.canvas.create_text(x + 7, y + 7, text=str(self.next_vertex_id), anchor="w", fill='red')
                self.new_point = Vertex(x, y, self.next_vertex_id)
                self.next_vertex_id += 1
                self.canvas.unbind("<Button-1>")
                self.canvas.quit()
            else:
                print('Clicked outside universal triangle')
        else:
            self.canvas.unbind("<Button-1>")
            self.canvas.quit()

    self.canvas.bind("<Button-1>", handle_click)
    self.canvas.mainloop()

def insert(self, p: Vertex, triangle_id: int):
    """ -----
    Nom: insert
    Auteur: Corinne Dumais
    Description: Insert le nouveau point dans le maillage.
    Date: 2023-13-03
    Version: 1.00
    ----- """

    s1, s2, s3 = self.TIN[triangle_id].vertices
    a1, a2, a3 = self.TIN[triangle_id].adjacents

    # Configuration des nouveaux triangles
    new_id1 = self.next_triangle_id
    new_id2 = self.next_triangle_id + 1
    self.next_triangle_id += 2

    self.TIN[triangle_id] = Triangle(triangle_id, [p, s2, s3], [a1, new_id1, new_id2])
    self.TIN[new_id1] = Triangle(new_id1, [p, s3, s1], [a2, new_id2, triangle_id])
    self.TIN[new_id2] = Triangle(new_id2, [p, s1, s2], [a3, triangle_id, new_id1])

    # On met à jour les triangles adjacents
    for a in [a1, a2, a3]:
        if a != 0:
            for i in [triangle_id, new_id1, new_id2]:
                if a in self.TIN[i].adjacents:
                    self.TIN[a].adjacents[self.TIN[a].adjacents.index(triangle_id)] = i

    # On ajoute les 3 nouveaux triangles à la pile
    self.stack.append(triangle_id)
    self.stack.append(new_id1)
    self.stack.append(new_id2)

    # On relie le nouveau point au sommet du triangle dans lequel il se trouve
    line_id1 = self.canvas.create_line(p.x, p.y, s1.x, s1.y)
    line_id2 = self.canvas.create_line(p.x, p.y, s2.x, s2.y)
    line_id3 = self.canvas.create_line(p.x, p.y, s3.x, s3.y)

    # On stocke les identificateurs des lignes pour pouvoir les effacer si nécessaire

```

```

self.lines[tuple(sorted([(p.x, p.y), (s1.x, s1.y)], key=lambda x: x[0]))] = line_id1
self.lines[tuple(sorted([(p.x, p.y), (s2.x, s2.y)], key=lambda x: x[0]))] = line_id2
self.lines[tuple(sorted([(p.x, p.y), (s3.x, s3.y)], key=lambda x: x[0]))] = line_id3

def optim(self):
    """ -----
    Nom: optim
    Auteur: Corinne Dumais
    Description: Optimise les triangles lorsque nécessaire jusqu'à ce que la pile soit vide
    Date: 2023-13-03
    Version: 1.00
    ----- """
    # On dépile tant qu'il y a des éléments dans la pile
    while self.stack:
        t1 = self.stack[-1]
        t2_idx = None
        for i, v in enumerate(self.TIN[t1].vertices):
            if v.number == self.new_point.number:
                t2_idx = i

        self.stack.pop()
        t2 = self.TIN[t1].adjacents[t2_idx]

        # Il n'y a pas de triangle de ce côté là (extérieur du triangle universel)
        if t2 == 0:
            continue

        pc = []
        for i, v in enumerate(self.TIN[t1].vertices):
            if v in self.TIN[t2].vertices:
                pc.append((i, v))

        t1_idx = self.TIN[t2].adjacents.index(t1)
        opposed_vertex = self.TIN[t2].vertices[t1_idx]

        # On vérifie si le triangle doit être optimisé
        if self.in_circle(self.TIN[t1], opposed_vertex):
            # Si oui, on inverse les diagonales
            pn, pnc, pc1, pc2 = self.swap(t1, t2)
            self.canvas.delete(self.lines[tuple(sorted([(pc1.x, pc1.y), (pc2.x, pc2.y)], key=lambda x: x[0]))])
            line_id = self.canvas.create_line(pn.x, pn.y, pnc.x, pnc.y, fill='black')
            self.lines[tuple(sorted([(pn.x, pn.y), (pnc.x, pnc.y)], key=lambda x: x[0]))] = line_id

def swap(self, t1: int, t2: int) -> Tuple[Vertex, Vertex, Vertex, Vertex]:
    """ -----
    Nom: swap
    Auteur: Corinne Dumais
    Description: Echange les diagonales de deux triangles dont les ID sont t1 et t2.
    Date: 2023-13-03
    Version: 1.00
    ----- """
    pc = []
    pn, pnc = None, None
    idx_pn = None
    for i, v in enumerate(self.TIN[t1].vertices):
        if v.number == self.new_point.number:
            pn = v
            idx_pn = i
        else:
            pc.append((i, v))

    assert len(pc) == 2, f"Two adjacent triangles should have exactly two vertices in common"

    if idx_pn == 1:
        pc1, pc2 = pc[1][1], pc[0][1]
        pc1_idx, pc2_idx = pc[1][0], pc[0][0]
    else:
        pc1, pc2 = pc[0][1], pc[1][1]

```

```

        pc1_idx, pc2_idx = pc[0][0], pc[1][0]

    t3 = self.TIN[t1].adjacents[pc1_idx]
    t4 = self.TIN[t1].adjacents[pc2_idx]

    pc1_idx, pc2_idx = None, None
    for i, v in enumerate(self.TIN[t2].vertices):
        if v.number in [n.number for n in self.TIN[t1].vertices]:
            if v.number == pc1.number:
                pc1_idx = i
            elif v.number == pc2.number:
                pc2_idx = i
            else:
                print('Vertex number does not correspond to one of the two common points')
        else:
            pnc = v

    t5 = self.TIN[t2].adjacents[pc2_idx]
    t6 = self.TIN[t2].adjacents[pc1_idx]

    # On écrase les définitions de T1 et T2 avec leur nouvelle configuration
    self.TIN[t1] = Triangle(t1, [pn, pnc, pc2], [t6, t3, t2])
    self.TIN[t2] = Triangle(t2, [pn, pc1, pnc], [t5, t1, t4])

    # On met à jour les triangles adjacents de T4 et T6
    if t4 != 0:
        self.TIN[t4].adjacents[self.TIN[t4].adjacents.index(t1)] = t2
    if t6 != 0:
        self.TIN[t6].adjacents[self.TIN[t6].adjacents.index(t2)] = t1

    # On remet les triangles dans la pile pour retester
    self.stack.append(t1)
    self.stack.append(t2)

    return pn, pnc, pc[0][1], pc[1][1]

@staticmethod
def in_circle(triangle: Triangle, point: Vertex) -> bool:
    """ -----
    Nom: in_circle
    Auteur: Corinne Dumais
    Description: Détermine si un point est dans le cercle englobant un triangle.
    Date: 2023-13-03
    Version: 1.00
    ----- """
    x, y = point.x, point.y
    x1, y1 = triangle.vertices[0].x, triangle.vertices[0].y
    x2, y2 = triangle.vertices[1].x, triangle.vertices[1].y
    x3, y3 = triangle.vertices[2].x, triangle.vertices[2].y

    z = x ** 2 + y ** 2
    z1 = x1 ** 2 + y1 ** 2
    z2 = x2 ** 2 + y2 ** 2
    z3 = x3 ** 2 + y3 ** 2

    H = np.linalg.det([[x, y, z, 1],
                       [x1, y1, z1, 1],
                       [x2, y2, z2, 1],
                       [x3, y3, z3, 1]])

    return H < 0

@staticmethod
def circumcircle(triangle: Triangle) -> Tuple[float, float]:
    """ -----
    Nom: circumcircle
    Auteur: Corinne Dumais
    Description: Calcule le centre circonscrit d'un triangle pour obtenir les vertex de Voronoï.
    Date: 2023-13-03
    ----- """

```

```

Version: 1.00
----- """
x1, y1 = triangle.vertices[0].x, triangle.vertices[0].y
x2, y2 = triangle.vertices[1].x, triangle.vertices[1].y
x3, y3 = triangle.vertices[2].x, triangle.vertices[2].y

z1 = x1 ** 2 + y1 ** 2
z2 = x2 ** 2 + y2 ** 2
z3 = x3 ** 2 + y3 ** 2

P = np.linalg.det([[y1, z1, 1], [y2, z2, 1], [y3, z3, 1]])
Q = -np.linalg.det([[x1, z1, 1], [x2, z2, 1], [x3, z3, 1]])
R = np.linalg.det([[x1, y1, 1], [x2, y2, 1], [x3, y3, 1]])

epsilon = 1E-4 # pour garantir la stabilité numérique
xc = -0.5 * P / (R + epsilon)
yc = -0.5 * Q / (R + epsilon)

return xc, yc

@staticmethod
def point_inside_universal(x: float, y: float, vertices: List[float]) -> bool:
    """
    -----
    Nom: point_inside_universal
    Auteur: Corinne Dumais
    Description: Détermine si un point est dans le triangle universel.
    Date: 2023-13-03
    Version: 1.00
    -----
    """
    x1, y1, x2, y2, x3, y3 = vertices
    # Calculate the area of the triangle and the areas of the sub-triangles formed by the point and each side of the triangle
    area = abs((x2 - x1) * (y3 - y1) - (x3 - x1) * (y2 - y1))
    area1 = abs((x1 - x) * (y2 - y) - (x2 - x) * (y1 - y))
    area2 = abs((x2 - x) * (y3 - y) - (x3 - x) * (y2 - y))
    area3 = abs((x3 - x) * (y1 - y) - (x1 - x) * (y3 - y))
    # The point is inside the triangle if the sum of the sub-triangle areas is equal to the area of the triangle
    return area == area1 + area2 + area3

def tracage_delaunay(self):
    """
    -----
    Nom: tracage_delaunay
    Auteur: Corinne Dumais
    Description: Prend des points fournis par l'utilisateur et fournit la triangulation Delaunay.
    Date: 2023-13-03
    Version: 1.00
    -----
    """
    self.delaunay = True
    self.sub_draw.entryconfigure(0, state='disabled')
    self.sub_draw.entryconfigure(1, state='normal')

    while self.delaunay:
        self.dessiner_nouveau_point()

        # On revérifie la condition car la valeur peut changer pendant qu'on attend un nouveau point dans draw_point
        if self.delaunay:
            id_triangle = self.walk(self.new_point)

            self.insert(self.new_point, id_triangle)

            self.optim()

    # Pour afficher les numéros des triangles dans chacun des triangles
    # Utile surtout pour débogage
    # if self.displayNumbers:
    #     for tid in self.text_ids:
    #         self.canvas.delete(tid)
    #
    # for i, t in self.TIN.items():

```

```

#         cx = (t.vertices[0].x + t.vertices[1].x + t.vertices[2].x) / 3
#         cy = (t.vertices[0].y + t.vertices[1].y + t.vertices[2].y) / 3
#
#         text_id = self.canvas.create_text(cx, cy, text=str(i), fill='blue')
#         self.text_ids.append(text_id)

def stop_delaunay(self):
    """ -----
    Nom: stop_delaunay
    Auteur: Corinne Dumais
    Description: Met fin à la triangulation Delaunay (après cet appel, l'utilisateur ne peut plus ajouter de point)
    Date: 2023-13-03
    Version: 1.00
    ----- """
    self.delaunay = False
    self.sub_draw.entryconfigure(1, state='disabled')
    self.triangulation_menu.entryconfigure(1, state='normal')
    self.analysis_menu.entryconfigure(0, state='normal')
    self.analysis_menu.entryconfigure(1, state='normal')

def tracage_voronoi(self):
    """ -----
    Nom: tracage_voronoi
    Auteur: Corinne Dumais
    Description: Trace le diagramme de Voronoi correspondant à la triangulation Delaunay.
    Date: 2023-13-03
    Version: 1.00
    ----- """
    for tid, triangle in self.TIN.items():
        center = self.circumcircle(triangle)
        self.TIN[tid].center = center

    for tid, triangle in self.TIN.items():
        xc, yc = triangle.center
        for a in triangle.adjacents:
            if a != 0:
                c = self.TIN[a].center
                self.canvas.create_line(xc, yc, c[0], c[1], fill='red')

def voisin_plus_proche(self):
    """ -----
    Nom: voisin_plus_proche
    Auteur: Corinne Dumais
    Description: Trouve et encercle le voisin le plus proche d'un point déterminé par l'utilisateur
    Date: 2023-13-03
    Version: 1.00
    ----- """
    def handle_click(event):
        if self.point_inside_universal(event.x, event.y, self.canvas.coords(self.universal)):
            x, y = event.x, event.y
            point = Vertex(x, y)
            self.canvas.create_oval(x - 3, y - 3, x + 3, y + 3, fill='green')
            self.canvas.unbind("<Button-1>")
            self.canvas.quit()

            tid = self.walk(point)
            dist = []
            for i, v in enumerate(self.TIN[tid].vertices):
                d = np.sqrt((point.x - v.x) ** 2 + (point.y - v.y) ** 2)
                dist.append((i, d))
            sorted_dist = sorted(dist, key=lambda k: k[1])
            v = self.TIN[tid].vertices[sorted_dist[0][0]]
            self.canvas.create_oval(v.x - 6, v.y - 6, v.x + 6, v.y + 6, outline='green', width=2)
        else:
            print('Clicked outside universal triangle')

    self.canvas.bind("<Button-1>", handle_click)
    self.canvas.mainloop()

```

```
def voisins_immediats(self):
    """ -----
    Nom: voisins_immediats
    Auteur: Corinne Dumais
    Description: Trouve et encercle tous les voisins immédiats d'un point identifié par l'utilisateur
    Date: 2023-13-03
    Version: 1.00
    ----- """
    point = simpdialog.askinteger("Voisins immédiats",
                                  f"Numéro du point (Entre 1 et {self.next_vertex_id - 1}): ", parent=self.root,
                                  minvalue=1,
                                  maxvalue=self.next_vertex_id - 1)
    for tid, triangle in self.TIN.items():
        if point in [n.number for n in triangle.vertices]:
            for t in triangle.vertices:
                if t.number != point:
                    self.canvas.create_oval(t.x - 6, t.y - 6, t.x + 6, t.y + 6, outline='blue', width=2)

if __name__ == '__main__':
    DelaunayTriangulationApp(height=700, width=700)
```